



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΒΑΣΕΙΣ ΔΕΔΟΜΕΝΩΝ 6ο ΕΞΑΜΗΝΟ ΡΟΗ Α
ΕΞΑΜΗΝΙΑΙΑ ΕΡΓΑΣΙΑ

ΚΕΜΑΝΕΣ ΠΑΝΑΓΙΩΤΗΣ: 03123045
ΗΛΙΟΠΟΥΛΟΣ ΔΗΜΗΤΡΙΟΣ: 03123080
ΚΛΑΨΟΠΟΥΛΟΣ ΓΙΩΡΓΟΣ:03123092

Περιεχόμενα:

1. <u>Περιγραφή της Οργάνωσης της Βάσης</u>	σελ. 3
2. <u>Processing of data</u>	σελ. 4
3. <u>Explain analyze</u>	σελ. 6
4. <u>Ευρετήρια</u>	σελ. 12
5. <u>QUERIES</u>	σελ. 16
6. <u>TRIGGERS</u>	σελ. 26
7. <u>User Interface</u>	σελ. 36

Περιγραφή της Οργάνωσης της Βάσης

Το νοσοκομείο μας διαθέτει μια οντότητα-πίνακα Προσωπικό (Staff) και τις υποκλάσεις (IS-A) προσωπικού: Γιατρός (Doctor), Νοσηλεύτες (Nurses), Διοικητικό Προσωπικό (Administrative). Όλα τα μέλη του προσωπικού διαθέτουν ξεχωριστό αριθμό προσωπικού staff_id ο οποίος τα ξεχωρίζει από τα υπόλοιπα μέλη. Τα μέλη του προσωπικού δεν διαγράφονται από την βάση για λόγους αρχείου αλλά αντίθετα χρησιμοποιείται το πεδίο working_status το οποίο δείχνει αν εργάζονται τώρα στο νοσοκομείο. Το νοσοκομείο έχει 15 τμήματα (οντότητα Departments) στα οποία ανήκουν τα μέλη του προσωπικού. Κάθε τμήμα έχει βάρδιες, η κάθε βάρδια Shift του νοσοκομείου έχει ξεχωριστό αριθμό shift_id. Οι βάρδιες συνδέονται με το προσωπικό (Staff) που τις εκτελεί μέσω της σχέσης duty_team. Οι κλίνες του νοσοκομείου έχουν αριθμό bed_number ο οποίος είναι μοναδικός στο νοσοκομείο και κάθε κλίνη ανήκει σε ένα τμήμα. Οι ασθενείς χαρακτηρίζονται στο νοσοκομείο μοναδικά μέσω του ΑΜΚΑ τους (και επίσης δεν επιτρέπεται η διαγραφή τους για λόγους ιστορικού). Οι ασθενείς, σύμφωνα και με την εκφώνηση φτάνουν στο νοσοκομείο όπου γίνεται η διαλογή (οντότητα Triage) από κάποιον Νοσηλευτή (συνδέεται με αυτόν με την σχέση “έκανε διαλογή”). Η διαλογή έχει επίσης μοναδικό triage_id. Τα συμπτώματα που καταγράφονται κατά την διαλογή αποθηκεύονται με το triage_id στον πίνακα Triage_Symptoms. Οι νοσηλίες (Hospitalization) στις οποίες μπορεί να οδηγήσει η διαλογή (συνδέονται μεταξύ τους με την σχέση) έχουν και αυτές μοναδικό αριθμό hospitalization_id. Όπως υποδηλώνει το όνομά η οντότητα Hospitalization είναι η καρδιά της νοσηλείας. Όλα τα ιατρικά δεδομένα τα οποία έχουν προκύψει από τα δοσμένα excel αρχεία είναι ξεχωριστές οντότητες-πίνακες. Έτσι, η νοσηλεία συνδέεται με τις διαγνώσεις με τις σχέσεις διάγνωση εισόδου και διάγνωση εξόδου. Κατά την νοσηλεία γίνονται:

- 1) Εργαστηριακές εξετάσεις οι οποίες συνδέονται τόσο με την νοσηλεία όσο και με τον γιατρό που την παρήγγειλε με τριαδική σχέση.
- 2) Ιατρικές πράξεις, που συνδέονται με την νοσηλεία με αντίστοιχη σχέση. Οι ιατρικές πράξεις είναι υπερκλάση των χειρουργικών επεμβάσεων (όπως προκύπτει από τα excel) οι οποίες συνδέονται και με τον κύριο χειρουργό (γιατρός-doctor) αλλά και με άλλα μέλη του προσωπικού που τις διενήργησαν.
- 3) Συνταγογράφηση η οποία είναι μία 4-way σχέση. Αυτό είναι πλεονασμός καθώς συνδέεται τόσο με τον ασθενή όσο και την νοσηλεία, καθώς και με το φάρμακο και τον γιατρό που το έγραψε. Τα πρώτα δύο προκύπτουν το ένα από το άλλο (σε συνδυασμό και με την ημερομηνία συνταγογράφησης), ωστόσο αποφασίσαμε να μεταφέρουμε τους ασθενείς (μέσω του ΑΜΚΑ) στην συνταγογράφηση ώστε να είναι ευκολότερος ο έλεγχος των αλλεργιών σε κάποιο φάρμακο.

Τα φάρμακα περιέχουν αλλεργικές ουσίες οι οποίες διαχωρίστηκαν με preprocessing που θα αναλυθεί παρακάτω οπότε φάρμακα και αλλεργικές ουσίες είναι ξεχωριστές οντότητες που συνδέονται με την σχέση “περιέχει” (contains). Με τις αλλεργικές ουσίες συνδέονται φυσικά και οι ασθενείς μέσω της σχέσης “αλλεργικός σε” (allergic to).

Τελος, υποστηρίζεται και η δυνατότητα να έχουμε φωτογραφίες για τις βασικές οντότητες (προσωπικό, τμήματα, αίθουσες/χειρουργεία) με τις αντίστοιχες σχέσεις και οντότητες.

Η μετατροπή του ER διαγράμματος σε σχεσιακό προκύπτει άμεσα μέσω των γνωστών κανόνων μετατροπής σχέσεων ανάλογα με τα cardinalities (1:N, N:M, 1:1) μέσω ξένων κλειδιών ή δημιουργίας νέων πινάκων για την αναπαράσταση της σχέσης. Αυτό φυσικά έπεται της διαδικασίας μετατροπής των μη δυαδικών σχέσεων σε δυαδικές.

Processing of data

Τα δεδομένα που αφορούν τα φάρμακα, την κωδικοποίηση ιατρικών πράξεων και των εργαστηριακών εξετάσεων, την λίστα Κλειστών Ενοποιημένων Νοσηλίων και τις διαγνώσεις μπήκαν στην βάση με χρήση LOAD DATA INFILE από αρχεία CSV.

Τα συγκεκριμένα αρχεία τροποποιήθηκαν κατάλληλα. Αφαιρέθηκαν τυχόν φωτογραφίες, links και ότι άλλη πληροφορία δεν ήταν σχετική με τα δεδομένα. Σε μερικά προστέθηκε η στήλη id για να διασφαλιστεί η σωστή εισαγωγή των δεδομένων σε περίπτωση που το auto increment δεν λόγω σφάλματος δεν ξεκινάει από το 1.

Τα δεδομένα που υπήρχαν στην ελληνική γλώσσα δεν άλλαξαν ως προς το περιεχόμενο τους αλλά μεταφράστηκαν σε greeklish με χρήση VBA script στο excel. Το script είναι διαθέσιμο στον φάκελο code

Στην βάση όλες οι δραστικές ουσίες συλλέχθηκαν από το φάρμακα με χρήση power query στο excel. Συγκεκριμένα πρώτα χωρίστηκαν όλες οι δραστικές ουσίες κάθε γραμμής με delimiters το κόμμα και το pipe("|") και μετά βγάλαμε τα duplicates.

Κάθε φάρμακο και κάθε δραστική ουσία έχουν δικό τους id τύπου integer. Για να αντιστοιχηθεί κάθε φάρμακο με κάθε δραστική έγινε η εξής διαδικασία:

- δημιουργήθηκε καινούργια προσωρινή βάση
- δημιουργήθηκαν πίνακες medication και active substance με attributes (product_name, active_substances) και (substance_id, ubstance_name) αντίστοιχα
- Έπειτα δημιουργήθηκε πίνακας medication_substance με attributes (medication_id, substance_id), το οποίο γεμίσαμε με το παρακάτω query:

```
INSERT IGNORE INTO medication_substance (  
    medication_id,  
    substance_id  
)  
SELECT  
    m.medication_id,  
    a.substance_id  
FROM medication AS m  
JOIN active_substance AS a  
    ON FIND_IN_SET(
```

```

        TRIM(a.substance_name),
        REPLACE(
            REPLACE(
                REPLACE(
                    REPLACE(m.active_substances, ' | ', ', '),
                    '|',
                    ',',
                ),
                ',',
                ' ',
            ),
            ' ',
            ''
        )
    ) > 0;

```

Για κάθε φάρμακο, το query μετατρέπει την λίστα των δραστικών ουσιών του σε συμβατή μορφή με την συνάρτηση FIND_IN_SET, αντικαθιστώντας τα pipe σύμβολα με κόμματα και αφαιρώντας περιττά κενά γύρω από τους delimiters. Μετά ελέγχει αν κάθε δραστική ουσία του πίνακα active_substance υπάρχει μέσα στην λίστα φαρμάκου ουσιών του φαρμάκου. Όταν βρεθεί αντιστοίχιση, εισάγεται στον πίνακα medication_substance το ζεύγος (medication_id, substance_id).

Το query τρέχει σωστά μόνο αν τα φάρμακα και οι δραστικές ουσίες εισαχθούν στην βάση με id που αντιστοιχεί στην σειρά του csv που βρίσκονται.

Όσον αφορά τα υπόλοιπα δεδομένα της βάσης, εισήχθησαν με την χρήση LLMs (chatgpt, codex) με συγκεκριμένα prompts.

Explain analyze

Query 6:

query 6:

```
SELECT h.department_name,  
       h.admission_date,  
       h.discharge_date,  
       h.admission_diagnosis,  
       d1.description,  
       h.discharge_diagnosis,  
       d2.description,  
       h.current_cost,  
       he.total_experience  
FROM HOSPITALIZATION h  
JOIN TRIAGE t ON h.triage_id = t.triage_id  
JOIN DIAGNOSES d1 ON h.admission_diagnosis = d1.diagnosis_code  
JOIN DIAGNOSES d2 ON h.discharge_diagnosis = d2.diagnosis_code  
LEFT JOIN EVALUATION_HOSPITALIZATION he ON h.hospitalization_id = he.hospitalization_id  
WHERE t.AMKA = 11438369194;
```

Στο query 6 ζητάμε για συγκεκριμένο ασθενή το ιστορικό νοσηλειών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής της κάθε νοσηλείας του.

Το αποτέλεσμα από το explain είναι :

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	t	ref	PRIMARY, AMKA	AMKA	8	const	602	Using index
1	SIMPLE	h	eq_ref	PRIMARY, triage_id, admission_diagnosis, discharge_diagnosis	triage_id	4	hopsital.t.triage_id	1	
1	SIMPLE	d2	eq_ref	PRIMARY	PRIMARY	1002	hopsital.h.discharge_diagnosis	1	
1	SIMPLE	he	eq_ref	PRIMARY	PRIMARY	4	hopsital.h.hospitalization_id	1	
1	SIMPLE	d1	eq_ref	PRIMARY	PRIMARY	1002	hopsital.h.admission_diagnosis	1	

Η στήλη id του explain δείχνει σε ποιο select ανήκει κάθε γραμμή του execution plan. Αφού όλες οι γραμμές έχουν “id=1” σημαίνει ότι όλοι οι πίνακες που χρησιμοποιούνται στο query ανήκουν στο ίδιο select.

Το select_type δείχνει τι είδους select κάνει αυτό το κομμάτι. Εφόσον το select_type είναι simple για όλες τις γραμμές μπορούμε να συμπεράνουμε ότι το query έχει μόνο ένα select και δεν υπάρχει κάποιο άλλο σε subquery.

Η στήλη table δείχνει ποιον πίνακα εξετάζει εκείνη την ώρα το execution plan. Η σειρά των γραμμών δείχνει την σειρά με την οποία ο optimiser σκοπεύει να διαβάσει ή να ενώσει του πίνακες. Οι πίνακες γράφονται με τα aliases οπότε στην δική μας περίπτωση η σειρά με την οποία εξετάστηκαν είναι η εξής:

- TRIAGE

- HOSPITALIZATION
- DIAGNOSES (για admission diagnosis)
- DIAGNOSES (για discharge diagnosis)
- EVALUATION_HOSPITALIZATION

Η στήλη type δείχνει τον τρόπο πρόσβασης στα δεδομένα. Ένα πιο “direct”

τρόπος πρόσβασης στα δεδομένα καθιστά πιο γρήγορη την αναζήτηση. Το ref στην βάση εμφανίζεται επειδή η βάση χρησιμοποιεί index για να βρει τα AMKA, το οποίο είναι λογικό αφού TRIAGE.AMKA είναι FK, αλλά το η βάση μπορεί να επιστρέψει πολλές γραμμές στο αποτέλεσμα. Στις υπόλοιπες γραμμές το type είναι eq_ref γιατί πάλι χρησιμοποιείται index αλλά η βάση επιστρέφει το πολύ μια γραμμή επειδή αυτά που ψάχνουμε είναι είτε primary keys η unique.

Η στήλη possible_keys δείχνει ποια indexes θα μπορούσε να χρησιμοποιήσει η βάση για τον πίνακα και η στήλη keys ποια τελικά χρησιμοποιήθηκαν. Για παράδειγμα στο πρώτο υποψήφια indexes είναι το primary και το AMKA, αλλά τελικά χρησιμοποιήθηκε το AMKA επειδή στο query έχουμε “WHERE AMKA = ...”.

Η στήλη key len δείχνει πόσα bytes του index χρησιμοποιήθηκαν στο συγκεκριμένο κομμάτι του query. Ένα index μπορεί να είναι φτιαγμένο σε παραπάνω από μια στήλες (composite index) και με το key_len βλέπουμε πόσο μεγάλο κομμάτι του index αξιοποιήθηκε από την βάση. Στην δική μας περίπτωση όλα τα keys είναι απλά οπότε μπορούμε γνωρίζοντας το domain field του κλειδιού να βρούμε το key_len. Για παράδειγμα το AMKA είναι BIGINT και γνωρίζουμε ότι ένας BIGINT χρειάζεται 8 bytes άρα μπορούμε να συμπεράνουμε ότι key_len = 8

Το ref δείχνει με τι συγκρίνεται το index (δηλαδή το key) που χρησιμοποιείται. Για παράδειγμα το AMKA στο TRIAGE έχει ref = const (constant) γιατί συγκρίνεται με σταθερή τιμή (“WHERE AMKA = 10061102069”). Στην δεύτερη γραμμή έχουμε key = triage_id και ref = hospital.t.triage_id σημαίνει ότι η βάση χρησιμοποιεί το index triage_id του hospitalization και με βάση αυτό ψάχνει την τιμή triage_id του πίνακα TRIAGE

Το query ξεκινάει από το TRIAGE (t) και χρησιμοποιεί index στο AMKA λόγω της ύπαρξης της συνθήκης “WHERE t.amka = ...” με αποτέλεσμα η βάση να μην χρειάζεται να κάνει full table scan για να βρει το αποτέλεσμα.

Η στήλη possible

Η στήλη rows δείχνει τον εκτιμώμενο αριθμό γραμμών που θα χρειαστεί να εξεταστούν από τον optimiser για το συγκεκριμένο βήμα. Στο πρώτη γραμμή το αποτέλεσμα είναι rows = 602 άρα με index AMKA, θα εξεταστούν περίπου 602 γραμμές του TRIAGE. Στις υπόλοιπες γραμμές έχουμε rows=1 επειδή type = eq_ref και επομένως κάθε γραμμή ενός πίνακα αντιστοιχίζεται το πολύ με μια γραμμή του επόμενου

Η στήλη extra μας δίνει επιπλέον πληροφορίες για το πως εκτελείται το συγκεκριμένο κομμάτι του query. Στο πρώτη γραμμή έχουμε extras = Using index γεγονός που επιβεβαιώνει κάτι που ήδη γνωρίζαμε: το TRIAGE χρησιμοποιεί INDEX AMKA για εκείνο το βήμα το query. Η απουσία στην στήλη extra για τις άλλες γραμμές δεν συνεπάγεται ότι δεν χρησιμοποιείται index. Απλά ο optimizer δεν έχει κάτι να σχολιάσει για το συγκεκριμένο βήμα

Force index

Τώρα αλλάζουμε στο query την γραμμή “FROM TRIAGE t” και την κάνουμε:

— FROM TRIAGE t FORCE INDEX (PRIMARY)

Το αποτέλεσμα του explain φαίνεται παρακάτω:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	d1	ALL	PRIMARY	NULL	NULL	NULL	10565	
1	SIMPLE	h	ref	PRIMARY, triage_id, admission_diagnosis, discharge_diagnosis	admission_diagnosis	1002	hospital.d1.diagnosis_code	2	
1	SIMPLE	t	eq_ref	PRIMARY	PRIMARY	4	hospital.h.triage_id	1	Using where
1	SIMPLE	he	eq_ref	PRIMARY	PRIMARY	4	hospital.h.hospitalization_id	1	
1	SIMPLE	d2	eq_ref	PRIMARY	PRIMARY	1002	hospital.h.discharge_diagnosis	1	

Αυτό είναι suboptimal γιατί αναγκάζουμε την βάση να προτιμήσει το primary key του triage δηλαδή το triage_id αντί για το AMKA στο οποίο βασίζεται η WHERE συνθήκη. Βλέποντας τώρα το explain φαίνεται ότι το query ξεκινάει με full table scan του πίνακα DIAGNOSES και μετά με βάση τις διαγνώσεις που διάβασε βρίσκει νοσηλείες που έχουν admission diagnosis ίδιο με τον DIAGNOSES.diagnosis_code. Στο προηγούμενο query δεν λάμβανε χώρα full table scan του πίνακα DIAGNOSES. Το query είχε γίνει με τέτοιο τρόπο ώστε να μην το χρειαστεί και όταν φτάναμε στο βήμα όπου έπρεπε να βρούμε το description της διάγνωσης εισαγωγής είχαμε ήδη τον κωδικό της απο το HOSPITALIZATION.

Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης

Για το εκτιμώμενο κόστος του query Q6 μπορούμε να δούμε το πίνακα rows του explain του απλού και το suboptimal query κάθε φορά και να προσθέσουμε μεταξύ τους τις εγγραφές κάθε γραμμής για να βρούμε μια προσέγγιση του αριθμού των γραμμών που θα αναζητήσει η βάση. Στο απλό query 6 το άθροισμα είναι 606 και στο δεύτερο τό άθροισμα είναι 10570. Άρα μπορούμε να δούμε ότι το suboptimal query έχει χειρότερο εκτιμώμενο κόστος από το optimal.

Παρακάτω απεικονίζεται αντίστοιχα ο πραγματικός χρόνος εκτέλεσης του query 6 και του suboptimal query 6 με την χρήση του force index:

Oncology	2017-04-12	2017-04-14	F42.9	Ideopsychanagkastiki diatarachi, mi kathorismeni
		F43	16846	Antidراسi se entono stress kai diataraches prosarmogis
Oncology	2017-05-10	2017-05-17	F43.8	Alles antidraseis se entono stress
		F43.9	11997	Antidراسi se sovaro stress, mi kathorismeni
Oncology	2017-06-09	2017-06-21	F44.2	Emvrontisia aposyndetikoy typoy
		F44.3	10357	Diataraches ekstasiakoy typoy kai diataraches katochis toy eaytoy
Oncology	2017-07-07	2017-07-10	F44.7	Miktes aposyndetikes [metatreptikes] diataraches
		F44.8	5062	Alles aposyndetikes [metatreptikes] diataraches
Oncology	2017-08-06	2017-08-14	F45.1	Adiaforopoiiti somatomorfi diatarachi
		F45.2	15128	Ypochondriaki diatarachi
Oncology	2017-09-03	2017-09-16	F45.9	Somatomorfi diatarachi, mi kathorismeni
		F48	3328	Alles neyrosikes diataraches
Oncology	2017-10-03	2017-10-07	F48.9	Neyrosiki diatarachi, mi kathorismeni
		F50	41579	Diataraches sti lipsi trofis
Oncology	2017-10-31	2017-11-09	F50.3	Atypi psychogenis voylimia
		F50.4	30725	Yperfagia syndyazomeni me alles psychologikes diataraches
602 rows in set (0.019 sec)				

Oncology	Diataraches sti lipsi trofis		41579		
Oncology	2017-10-31	2017-11-09	F50.3	Atypi psychogenis voylimia	30725
Pediatrics	Yperfagia syndyazomeni me alles psychologikes diataraches				
	2024-10-12	2024-10-15	M66.9	Mesi otitida, mi kathorismeni	1082
Gastroenterology	Mesi otitida, mi kathorismeni				
	2026-01-22	2026-02-03	K80.2	Lithoi tis cholidochoy kystis choris cholokystitida	2039
Anesthesiology	Lithoi tis cholidochoy kystis choris cholokystitida				
	2018-01-13	2018-01-19	7A00	Cholera	2712
Anesthesiology	Cholera apo donakio tis choleras [Vibrio cholerae] 01, viotypo cholerae				
	2010-01-19	2010-01-25	7A00	Cholera	2712
Anesthesiology	Cholera apo donakio tis choleras [Vibrio cholerae] 01, viotypo cholerae				
602 rows in set (0.443 sec)					

Το κανονικό query τρέχει σε 0.019sec και το suboptimal τρέχει σε 0.443sec. Αρά μπορούμε να δούμε ότι το suboptimal query τρέχει σε περισσότερο χρόνο από το απλό.

Query 4

```
SELECT s.AMKA,
       s.last_name,
       s.first_name,
       (
         SELECT AVG(evaluation)
         FROM DOCTOR_EVALUATION de
         WHERE de.doctor_id = s.staff_id
       )AS average_doctor_evaluation,
       (
         SELECT AVG(total_experience)
         FROM EVALUATION_HOSPITALIZATION eh
         WHERE eh.hospitalization_id IN (
           SELECT hp.hospitalization_id AS hospitalization_id
           FROM SURGERY sur
           NATURAL JOIN HOSPITALIZATION_PROCEDURE hp
           WHERE sur.surgeon_id = s.staff_id

         UNION

         SELECT hospitalization_id AS hospitalization_id
         FROM HOSPITALIZATION_TEST
         WHERE s.staff_id = staff_id

         UNION

         SELECT hospitalization_id AS hospitalization_id
         FROM PRESCRIPTION
         WHERE s.staff_id = staff_id
       )
       ) AS average_total_experience
FROM STAFF s
WHERE s.staff_id = '5'
```

Αρχικά τρέχουμε το query στην αρχική του μορφή κάνοντας explain οπότε προκύπτει το παρακάτω πλάνο εκτέλεσης:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	s	const	PRIMARY	PRIMARY	4	const	1	
3	DEPENDENT SUBQUERY	eh	ALL	NULL	NULL	NULL	NULL	59151	Using where
4	DEPENDENT SUBQUERY	sur	ref	surgeon_id	surgeon_id	4	const	1	Using index
4	DEPENDENT SUBQUERY	hp	ref	hospitalization_id	hospitalization_id	4	func	1	Using index
5	DEPENDENT UNION	HOSPITALIZATION_TEST	ref	hospitalization_id,staff_id	hospitalization_id	4	func	1	Using where
6	DEPENDENT UNION	PRESCRIPTION	ref	PRIMARY,staff_id,AMKA,hospitalization_id	hospitalization_id	4	func	1	Using where; Using index
NULL	UNION RESULT	<union4,5,6>	ALL	NULL	NULL	NULL	NULL	NULL	
2	SUBQUERY	de	ref	PRIMARY	PRIMARY	4	const	1	

8 rows in set (0.044 sec)

Παρατηρούμε πως εκτός από τον υπολογισμό του IN κατά τον οποίο γίνεται Full Table scan για γίνει έλεγχος αν το αποτέλεσμα του union βρίσκεται στον πίνακα αυτόν, σε κάθε άλλη περίπτωση έχουμε την χρήση ευρετηρίου όπως υποδηλώνει το “ref” στο πεδίο type.

Αρχικά προσπαθήσαμε να κάνουμε FORCE INDEX (PRIMARY) στον πίνακα EVALUATION_HOSPITALIZATION για να επιταχύνουμε την εκτέλεση παρόλα αυτά το query plan δεν άλλαξε. Έπειτα του δώσαμε hint:

FROM DOCTOR_EVALUATION IGNORE INDEX (PRIMARY)

Στην περίπτωση αυτή προκύπτει το ακόλουθο query plan όταν τρέχουμε explain:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	s	const	PRIMARY	PRIMARY	4	const	1	
3	DEPENDENT SUBQUERY	eh	ALL	NULL	NULL	NULL	const	60676	Using where
4	DEPENDENT SUBQUERY	sur	ref	surgeon_id	surgeon_id	4	const	1	Using index
4	DEPENDENT SUBQUERY	hp	ref	hospitalization_id	hospitalization_id	4	func	1	Using index
5	DEPENDENT UNION	HOSPITALIZATION_TEST	ref	hospitalization_id,staff_id	hospitalization_id	4	func	1	Using where
6	DEPENDENT UNION	PRESCRIPTION	ref	PRIMARY,staff_id,AMKA,hospitalization_id	hospitalization_id	4	func	1	Using where; Using index
NULL	UNION RESULT	<union4,5,6>	ALL	NULL	NULL	NULL	NULL	NULL	
2	SUBQUERY	de	ALL	NULL	NULL	NULL	NULL	200	Using where

8 rows in set (0.081 sec)

Τώρα, ενώ οι υπόλοιπες πράξεις υλοποιούνται με τον ίδιο τρόπο στο Subquery με id=2 ο query optimiser αναγκάζεται να εκτελέσει Full table scan όπως υποδηλώνει το “ALL” στο αντίστοιχο πεδίο type. Έτσι αλλάζει και το πλήθος των tuples-rows τις οποίες θα πρέπει να προσπελάσει και από 1 γίνεται 200.

Εκτελούμε πολλαπλές φορές το query χωρίς και το query με το hint και προκύπτει πως οι πιο αντιπροσωπευτικοί χρόνοι εκτέλεσης είναι οι ακόλουθοι:

- Αρχικό query: 0,815 sec

AMKA	last_name	first_name	average_doctor_evaluation	average_total_expierience
44613566954	Beecker	Latrena	3.0000	4.0000

1 row in set (0.815 sec)

- Suboptimal query με IGNORE: 0,846 sec

AMKA	last_name	first_name	average_doctor_evaluation	average_total_expierience
44613566954	Beecker	Latrena	3.0000	4.0000

1 row in set (0.846 sec)

Σύμφωνα με το αποτέλεσμα των explain εάν θεωρήσουμε, όπως και παραπάνω, πως ο αριθμός των rows που ελέγχει το RDBMS είναι ανάλογος του χρόνου εκτέλεσης τότε έχουμε ότι:

- Αρχικό Query: $r_1 = \#rows = 60676 + 1 + 1 + 1 + 1 + 1 = 60681$
- Suboptimal Query: $r_2 = \#rows = 60676 + 1 + 1 + 1 + 1 + 200 = 60880$

Αρα επιβεβαιώνονται τα αποτελέσματα της χρονικής σύγκρισης που παρατέθηκαν παραπάνω.

Όπως αναφέραμε και όταν εξετάσαμε τα query plans που προέκυψαν με και χωρίς hint η διαφορά στην εκτέλεση, η οποία καθιστά πιο αργή την δεύτερη εκδοχή του query (με το ignore) είναι πως αναγκάζεται να μην χρησιμοποιήσει το B+ ευρετήριο το οποίο έχει ορίσει η ίδια η MySQL για το PK του πίνακα DOCTOR_EVALUATION με αποτέλεσμα να εκτελεί full table scan το οποίο οδηγεί σε περαιτέρω καθυστέρηση. Έτσι, αντί να έχουμε έλεγχο πρακτικά σε ένα row (const) πρέπει να κοιτάζουμε και τα 200 tuples του πίνακα. Φυσικά το μεγαλύτερο τμήμα του χρόνου αφιερώνεται στον έλεγχο των tuples που προκύπτουν από το UNION ώστε να προσδιοριστούν αυτά που έχουν HOSPITALIZATION EVALUATION.

Ευρετήρια

Οι ορισμοί ευρετηρίων γίνονται για να επιταχυνθούν τα ερωτήματα (queries) τα οποία διαχειρίζονται πίνακες με αρκετά δεδομένα αλλά και τα triggers, ιδίως αυτά τα οποία καλούνται κατά το insert. Για αυτό για να προβούμε στην δημιουργία των indices πρέπει πρώτα να εντοπίσουμε τις περιστάσεις σε queries (κυρίως) αλλά και triggers που έχουμε equi-joins, αναζητήσεις (WHERE) και aggregations (GROUP BY). Φυσικά πρέπει να είμαστε προσεκτικοί στις επιλογές μας ώστε να μην επιβαρύνεται χωρίς λόγο το DBMS προσπαθώντας να διατηρήσει ένα B+ δέντρο κάθε φορά που γίνεται μία νέα εγγραφή.

Για να επιλέξουμε σε ποιους πίνακες και συγκεκριμένα σε ποια attributes πρέπει να δημιουργηθούν ευρετήρια αναλύουμε διαδοχικά, για τα queries:

- Equi - Joins
- Where Clauses
- Group By Aggregation

Equi - Joins

Συχνότητα	Πίνακας & Πεδίο	Queries που εμφανίζεται
5 φορές	HOSPITALIZATION.triage_id & TRIAGE.triage_id	Q1, Q3, Q6, Q9, Q14, Q15
3 φορές	PATIENTS.AMKA & TRIAGE.AMKA	Q1, Q3, Q9
3 φορές	STAFF.staff_id & DOCTOR.staff_id	Q2, Q5, Q11
3 φορές	SHIFTS.shift_id & DUTY_TEAM.shift_id	Q2, Q8, Q12
3 φορές	ACTIVE_SUBSTANCE.substance_id & MEDICATION_SUBSTANCE.substance_id	Q7, Q10 (δύο φορές)
2 φορές	SURGERY.surgery_id & HOSPITALIZATION_PROCEDURE.procedure_id	Q4, Q11
2 φορές	DUTY_TEAM.staff_id & STAFF.staff_id	Q8, Q12
2 φορές	PRESCRIPTION.medication_id & MEDICATION_SUBSTANCE.medication_id	Q10
1 φορά	HOSPITALIZATION.admission_diagnosis, discharge_diagnosis	Q6

Τα join είναι οι βαρύτερες πράξεις και για αυτό πρέπει να επιταχυνθούν χρησιμοποιώντας indexed loop join (έναντι του block nested loop join). Τα Primary Keys ευρετηριάζονται αυτόματα από την MySQL. Επιπλέον, η MySQL “μεριμνά” (έμμεσα) για τα συχνότερα join τα οποία αφορούν Foreign Keys κάνοντας δημιουργώντας ευρετήρια σε όλα τα Foreign Keys κάθε πίνακα. Επομένως, ενώ θα έπρεπε εμείς να προσθέσουμε ευρετήρια για την γρηγορότερη διεξαγωγή των join δεν είναι αυτό απαραίτητο καθώς συμβαίνει ήδη.

Εάν έπρεπε να οριστούν από εμάς θα έπρεπε να δημιουργηθούν σίγουρα για τα ακόλουθα πεδία:

- HOSPITALIZATION.triage_id
- TRIAGE.AMKA
- DUTY_TEAM.shift_id (το ευρετήριο στο composite Primary Key (staff_id, shift_id) δεν μπορεί να χρησιμοποιηθεί για αναζήτηση του shift_id)
- MEDICATION_SUBSTANCE.substance_id (όμοια με το προηγούμενο καθώς δεν αρκεί το index στο composite PK)
- PRESCRIPTION.medication_id

Αντίστοιχα κοιτάμε κατηγορήματα ισότητας που εμφανίζονται σε WHERE:

Where

Συχνότητα	Πίνακας & Πεδίο	Queries που εμφανίζεται
3 φορές	SHIFTS.shift_date	Q2, Q8, Q12
2 φορές	HOSPITALIZATION.discharge_date	Q1, Q9
2 φορές	STAFF.staff_id	Q2, Q4
2 φορές	SURGERY.surgeon_id	Q2, Q4
1 φορά	HOSPITALIZATION_PROCEDURE.procedure_date	Q11
1 φορά	TRIAGE.AMKA	Q6
1 φορά	DOCTOR.specialty	Q2
1 φορά	STAFF.age	Q5
1 φορά	SHIFTS.department_name	Q8

Όμοια με παραπάνω τα περισσότερα είναι FK οπότε είναι ευρετηριασμένα ήδη από την MySQL. Από όλες τις περιπτώσεις που εμφανίζονται περισσότερες από μία φορές μόνο το πεδίο SHIFTS.shift_date (το οποίο χρησιμοποιείται και σε πολλά triggers όπως θα δούμε στην συνέχεια) και το HOSPITALIZATION.discharge_date δεν έχει ευρετήριο.

Για τα aggregate functions έχουμε αντίστοιχα:

Group By

Συχνότητα	Πίνακας & Πεδίο	Queries που εμφανίζεται
3 φορές	HOSPITALIZATION.admission_date	Q1, Q9, Q14 (Συνήθως ως YEAR())
2 φορές	HOSPITALIZATION.department_name	Q1, Q3
2 φορές	PATIENTS.AMKA	Q3, Q9
2 φορές	ACTIVE_SUBSTANCE.substance_id	Q7, Q10
1 φορά	SHIFTS.shift_date, slot, department_name	Q12
1 φορά	HOSPITALIZATION.drg_code	Q1
1 φορά	TRIAGE.urgency_level	Q15

Και εδώ τα πεδία που συναντούμε είναι ήδη ευρετηριασμένα και παρατηρούμε ότι εμφανίζεται ακόμη μία φορά το SHIFT.shift_date και το SHIFT.slot. Επιπλέον ωστόσο παρατηρούμε την εμφάνιση του πεδίου HOSPITALIZATION.admission_date που χρησιμοποιείται για queries που αναζητούν δεδομένα ανά έτος. Επιπλέον εμφανίζεται και το non-FK πεδίο TRIAGE.urgency_level.

Επαναλαμβάνουμε τον έλεγχο για τα JOIN και WHERE των triggers.

Join

Συχνότητα	Πίνακας & Πεδίο	Παρατηρήσεις
5 φορές	DUTY_TEAM.shift_id = SHIFTS.shift_id	Ελέγχει συνεχώς τα όρια βαρδιών, τα ωράρια ξεκούρασης και τα ρεπό.
2 φορές	DUTY_TEAM.staff_id = STAFF.staff_id	Για επιβεβαίωση ρόλων κάθε βάρδιας.
2 φορές	DUTY_TEAM.staff_id = DOCTOR.staff_id	Για επιβεβαίωση αν υπάρχει ειδικευόμενος / επιβλέπων.
1 φορά	ms.substance_id = ap.substance_id, h.triage_id = t.triage_id	Συνδέσεις για Αλλεργίες / Αξιολογήσεις

WHERE

Συχνότητα	Πίνακας & Πεδίο	Παρατηρήσεις
4 φορές	DUTY_TEAM.staff_id	Η βάση ψάχνει συνεχώς "Τι άλλες βάρδιες έχει κάνει αυτός ο υπάλληλος;"
2 φορές	DUTY_TEAM.shift_id	Η βάση αναζητά "Ποιοι δουλεύουν σε αυτή τη βάρδια;"
2 φορές	SHIFTS.shift_date	Γίνεται έλεγχος ημερομηνιών για συνεχόμενες νύχτες και όρια μήνα.
1 φορά	PRESCRIPTION.staff_id & AMKA	Για να επιβεβαιώσει αν ο ασθενής έχει δικαίωμα να αξιολογήσει τον γιατρό.
1 φορά	MEDICATION_SUBSTANCE.medicati on_id	Στον έλεγχο αλλεργιών.

Και εδώ, στις περισσότερες περιπτώσεις οι αναζητήσεις και τα join σε ήδη ευρετηριασμένα ξένα κλειδιά με το πεδίο SHIFT.shift_date να είναι το μόνο που δεν είναι FK.

Από την παραπάνω ανάλυση προκύπτει ότι νέα ευρετήρια χρειάζεται να ορίσουμε είναι:

- Composite ευρετήριο στα πεδία (shift_date, slot) του SHIFTS το οποίο όπως γνωρίζουμε μπορεί να χρησιμοποιηθεί και ως ευρετήριο για το shift_date και χρειάζεται σε queries αλλά και πολλά triggers, δηλαδή:

```
CREATE INDEX idx_shifts_date_slot ON SHIFTS(shift_date, slot);
```

- Ευρετήριο στο TRIAGE.urgency_level λόγω του μεγέθους του πίνακα για το query 15, δηλαδή:

```
CREATE INDEX idx_triage_urgency ON TRIAGE(urgency_level);
```

- Composite ευρετήριο στα πεδία (service_time, urgency_level, arrival_time) επίσης στο TRIAGE καθώς έχουμε ορίσει διαδικασία η οποία βρίσκει τους επόμενους ασθενείς που πρέπει να εξυπηρετήσουμε βάση της ουράς προτεραιότητας. Αυτό είναι χρήσιμο καθώς εκτός από το μέγεθος του πίνακα, συμβαίνει και πολύ συχνά. Οπότε έχουμε:

```
CREATE INDEX idx_triage_waiting ON  
TRIAGE(service_time, urgency_level, arrival_time);
```

QUERIES

Σε αυτό το σημείο της εργασίας καλούμαστε να δημιουργήσουμε 15 queries που θα επιστρέφουν αποτελέσματα βασισμένα στα δεδομένα που εισαγάγαμε στην βάση μας. Οι υλοποιήσεις μας για τα ζητούμενα queries παρουσιάζονται παρακάτω:

1)

```
SELECT YEAR(h.admission_date) AS year,
       h.department_name AS department_name,
       h.drg_code AS KEN,
       p.insurance_provider AS insurance_provider,
       COUNT(*) AS hospitalization_count,
       COUNT(*) *
       (
         SELECT cost
         FROM DRG
         WHERE drg_code = h.drg_code
       ) AS base_cost,
       SUM(current_cost) - COUNT(*) *
       (
         SELECT cost
         FROM DRG
         WHERE drg_code = h.drg_code
       ) AS extra_cost,
       SUM(current_cost) AS Total_Cost
FROM hospitalization h
JOIN triage AS t ON h.triage_id = t.triage_id
JOIN patients p ON p.AMKA = t.AMKA
WHERE h.discharge_date IS NOT NULL
GROUP BY YEAR(h.admission_date),
         h.department_name,
         h.drg_code,
         p.insurance_provider;
```

Εδώ θέλουμε να βρούμε τα συνολικά έσοδα του νοσοκομείου ανά έτος, τμήμα, κωδικό KEN και ασφαλιστικό.

Στο query βρίσκουμε το base_cost ως το κόστος που προκύπτει από το κόστος του KEN κάθε νοσηλείας και το υπολογίζουμε ως το γινόμενο του αριθμού των νοσηλείων και του κόστους KEN της κάθε μίας. Το current_cost που αναφέρεται στον κώδικα προκύπτει από trigger κατά την εισαγωγή της εκάστοτε νοσηλείας και έτσι υπολογίζουμε το extra_cost ως την διαφορά του συνολικού κόστους της κάθε νοσηλείας με το βασικό κόστος της. Τα δύο joins έχουν τοποθετηθεί ώστε να συνδέσουν τον πίνακα των νοσηλείων με τους ασθενείς.

2)

```
SELECT s.last_name,
       s.first_name,
       s.AMKA,

       CASE
         WHEN EXISTS
           (
             SELECT 1
             FROM SHIFTS sh
             JOIN DUTY_TEAM dt ON dt.shift_id = sh.shift_id
             WHERE s.staff_id = dt.staff_id
             AND YEAR(sh.shift_date) = YEAR(CURRENT_DATE)
             AND sh.shift_date < CURRENT_DATE
             AND sh.shift_status = "full"
           )
         THEN 'YES'
         ELSE 'NO'
       END AS shift_current_year,

       (
         SELECT COUNT(*)
         FROM SURGERY
         WHERE surgeon_id = s.staff_id
       )
       AS surgery_count
FROM staff s
JOIN doctor d ON s.staff_id = d.staff_id
WHERE d.specialty = "Neurosurgeon";
```

Στο συγκεκριμένο query θέλουμε να βρούμε τους γιατρούς μίας συγκεκριμένης ειδικότητας με ένδειξη για το αν έχουν μια τουλάχιστον εφημερία το τρέχον έτος και το αν πόσες επεμβάσεις έχουν εκτελέσει ως κύριοι χειρουργοί.

Πιο συγκεκριμένα βρίσκουμε για όλους τους γιατρούς εάν έχουν κάνει βάρδια το τρέχον έτος και θέτουμε το shift_current_year ως YES ή NO (χρησιμοποιώντας το 'CASE'). Μετά ελέγχουμε τον αριθμό των χειρουργείων που έχει πραγματοποιήσει ο εκάστοτε γιατρός ως κύριος χειρουργός. Το join γίνεται έτσι ώστε να συνδέσει τον πίνακα doctor με τον πίνακα staff. Επιλέγουμε αυθαίρετα την ειδικότητα "Cardiology". Σημειώνουμε ότι σύμφωνα με περιορισμό οι γιατροί που έχουν αυτή την ειδικότητα ΔΕΝ μπορούν να διενεργήσουν χειρουργικές επεμβάσεις (README).

3)

```
SELECT
  p.AMKA,
  SUM(h.current_cost) AS total_cost,
  h.department_name

FROM patients p
JOIN triage t ON p.AMKA=t.AMKA
JOIN hospitalization h ON h.triage_id=t.triage_id
GROUP BY p.AMKA, h.department_name
HAVING COUNT(*)>3
```

Με αυτό το query θέλουμε να βρούμε ποιού ασθενείς έχουν νοσηλευτεί τρεις ή περισσότερες φορές σε ένα τμήμα καθώς και το συνολικό κόστος των νοσηλείων τους.

Στην υλοποίηση μας το συνολικό κόστος υπολογίζεται από το SUM(current_cost) το οποίο περιλαμβάνει τόσο το βασικό κόστος όσο και extra_cost της κάθε νοσηλείας. Τα δύο joins

γίνονται και πάλι έτσι ώστε να συνδεθεί ο πίνακας των ασθενών με τις νοσηλείες. Τέλος τα αποτελέσματα γίνονται group by ανά ασθενή και τμήμα κρατώντας μόνο όσους συνδυασμούς έχουν περισσότερα από 3 tuples.

4)

```
SELECT s.AMKA,
       s.last_name,
       s.first_name,
       (
         SELECT AVG(evaluation)
         FROM DOCTOR_EVALUATION de
         WHERE de.doctor_id = s.staff_id
         )AS average_doctor_evaluation,
       (
         SELECT AVG(total_expierience)
         FROM EVALUATION_HOSPITALIZATION eh
         WHERE eh.hospitalization_id IN (
           SELECT hp.hospitalization_id AS hospitalization_id
           FROM SURGERY sur
           NATURAL JOIN HOSPITALIZATION_PROCEDURE hp
           WHERE sur.surgeon_id = s.staff_id

           UNION

           SELECT hospitalization_id AS hospitalization_id
           FROM HOSPITALIZATION_TEST
           WHERE s.staff_id = staff_id

           UNION

           SELECT hospitalization_id AS hospitalization_id
           FROM PRESCRIPTION
           WHERE s.staff_id = staff_id
         )
       ) AS average_total_expierience
FROM STAFF s
WHERE s.staff_id = '5'
```

Με αυτό το query θέλουμε να βρούμε για κάποιον γιατρό τον μέσο όρο των αξιολογήσεων των ασθενών του και την συνολική εμπειρία τους από την νοσηλεία.

Όπως φαίνεται, υπολογίζουμε τον μέσο όρο μέσω του πίνακα doctor_evaluation και την συνολική εμπειρία από το επόμενο κομμάτι του κώδικα. Εκεί εντοπίζουμε όλες τις διαφορετικές νοσηλείες με τις οποίες "αλληλεπιδράσε" ο συγκεκριμένος γιατρός με τον οποιοδήποτε τρόπο. Για αυτό τον λόγο κάνουμε union μεταξύ subqueries που βρίσκουν όλα τα hospitalization_ids που είτε έχει χειρουργήσει, είτε έχει κάνει κάποια εξέταση, είτε έχει συνταγογραφήσει ο εν λόγω γιατρός. Παίρνουμε την μέση τιμή των αξιολογήσεων όλων αυτών των νοσηλειών.

5)

```
SELECT d.staff_id, st.first_name, st.last_name, COUNT(*) AS total_surgeries
FROM doctor d
JOIN staff st ON st.staff_id=d.staff_id
JOIN surgery s ON s.surgeon_id=d.staff_id
WHERE st.age<35
GROUP BY d.staff_id
ORDER BY COUNT(*) DESC
```

Με το παραπάνω query θέλουμε να βρούμε τους γιατρούς με ηλικία κάτω των 35 ετών οι οποίοι έχουν διεξάγει τις περισσότερες επεμβάσεις ως κύριοι χειρουργοί. Πιο συγκεκριμένα ενώνουμε τον πίνακα των γιατρών με το staff και των χειρουργών με των γιατρών με την χρήση των δύο joins. Φιλτράρουμε τον πίνακα με τον παράγοντα της ηλικίας το αποτέλεσμα ταξινομείται με βάση το ποιός έχει κάνει τις περισσότερες επεμβάσεις ως κύριος χειρουργός.

6)

```
SELECT h.department_name,
       h.admission_date,
       h.discharge_date,
       h.admission_diagnosis,
       d1.description,
       h.discharge_diagnosis,
       d2.description,
       h.current_cost,
       he.total_experience
FROM HOSPITALIZATION h
JOIN TRIAGE t ON h.triage_id = t.triage_id
JOIN DIAGNOSES d1 ON h.admission_diagnosis = d1.diagnosis_code
JOIN DIAGNOSES d2 ON h.discharge_diagnosis = d2.diagnosis_code
JOIN EVALUATION_HOSPITALIZATION he ON h.hospitalization_id = he.hospitalization_id
WHERE t.AMKA = 10061102069
```

Με αυτό το query θέλουμε για έναν συγκεκριμένο ασθενή να βρούμε το ιστορικό νοσηλείων του, με τις αντίστοιχες διαγνώσεις, το συνολικό κόστος ανά νοσηλεία και το μέσο όρο των αξιολογήσεων του.

Για αυτό τον αναζητούμε στον πίνακα hospitalization όλα τα παραπάνω attributes. Κάνουμε δύο φορές join με τον πίνακα των διαγνώσεων έτσι ώστε να το συνδέσουμε τόσο με την διάγνωση εξόδου όσο και με την διαγνωση εισόδου. Ο μέσος όρος των αξιολογήσεων του ασθενούς είναι φυσικά ένας και εμφανίζεται (ίδιος) σε όλα τα tuples που επιστρέφονται.

7)

```
SELECT act.substance_name,
       COUNT(ap.AMKA) AS num_of_allergic_patients,
       (
         SELECT COUNT(*)
         FROM MEDICATION_SUBSTANCE
         WHERE substance_id = act.substance_id
       ) AS num_of_medication_with_substance
FROM ALLERGIC_PATIENTS AS ap
RIGHT OUTER JOIN ACTIVE_SUBSTANCE AS act ON ap.substance_id = act.substance_id
GROUP BY act.substance_id
ORDER BY num_of_allergic_patients DESC
```

Εδώ θέλουμε να βρούμε για κάθε δραστική ουσία τον αριθμό των αλλεργικών ασθενών που την έχουν δηλώσει και τον αριθμό φαρμάκων που την περιέχουν ταξινομημένα με βάση τον αριθμό των ασθενών.

Στην υλοποίηση μας παίρνουμε από το count(p.AMKA) τον αριθμό των αλλεργικών ασθενών που έχουν δηλώσει αλλεργία σε κάποια ουσία ενώ με το subquery βρίσκουμε σε συνδυασμό με το join τον αριθμό των φαρμάκων που περιέχουν την εκάστοτε ουσία. Στην συγκεκριμένη περίπτωση κάνουμε right outer join καθώς παρά την συνθήκη θέλουμε να κάνουμε την διαδικασία για κάθε active_substance, όχι μόνο για αυτά που έχουν δηλωθεί από κάποιον ασθενή.

8)

```
SET @dept_name = 'Cardiology';
SET @shift_day = '2026-05-10';

SELECT s.AMKA,
       s.last_name,
       s.first_name,
       s.staff_type
FROM (
    SELECT dd.staff_id AS staff_id
    FROM DOCTOR_DEPARTMENT dd
    WHERE dd.department_name = @dept_name

    UNION

    SELECT n.staff_id
    FROM NURSES n
    WHERE n.department_name = @dept_name

    UNION

    SELECT adm.staff_id
    FROM ADMINISTRATIVE_STAFF adm
    WHERE adm.department_name = @dept_name
) AS department_staff_id
JOIN STAFF s ON s.staff_id = department_staff_id.staff_id
WHERE NOT EXISTS(
    SELECT 1
    FROM DUTY_TEAM dt
    JOIN SHIFTS sh ON dt.shift_id = sh.shift_id
    WHERE s.staff_id = dt.staff_id
    AND sh.shift_date = @shift_day
    AND sh.department_name = @dept_name
);
```

Σε αυτό το query βρίσκουμε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

Με το (... UNION ... UNION ...) βρίσκουμε όλο το προσωπικό συγκεκριμένου τμήματος και έπειτα με το join παίρνουμε επώνυμο, όνομα, AMKA τον τύπο του προσωπικού αυτών των ανθρώπων. Τέλος με την χρήση του WHERE NOT EXISTS(...) φιλτράρουμε τους ανθρώπους που είχαν βάρδια στην επιθυμητή ημερομηνία.

9)

```
WITH Patient_Yearly_Stay AS (
    SELECT
        YEAR(h.admission_date) AS year,
        p.first_name,
        p.last_name,
        p.AMKA,
        SUM(DATEDIFF(h.discharge_date, h.admission_date)) AS total_days
    FROM PATIENTS p
    JOIN TRIAGE t ON p.AMKA = t.AMKA
    JOIN HOSPITALIZATION h ON t.triage_id = h.triage_id
    WHERE h.discharge_date IS NOT NULL
    GROUP BY YEAR(h.admission_date), p.AMKA, p.first_name, p.last_name
    HAVING SUM(DATEDIFF(h.discharge_date, h.admission_date)) > 15
),
Same_Durations AS (
    SELECT YEAR, total_days
    FROM Patient_Yearly_Stay
    GROUP BY YEAR, total_days
    HAVING COUNT(*) > 1
)
SELECT p1.*
FROM Patient_Yearly_Stay p1
JOIN Same_Durations p2 ON p1.YEAR = p2.YEAR AND p1.total_days = p2.total_days
ORDER BY p1.YEAR, p1.total_days, p1.last_name;
```

Τώρα θέλουμε να βρούμε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διαστημα ενός έτους με συνολική διάρκεια νοσηλείας άνω των 15 ημερών. Επομένως στο πρώτο σκέλος του κώδικα βρίσκουμε τους ασθενείς οι οποίοι έχουν αριθμό νοσηλειών μεγαλύτερο από 15 μέσα σε ένα έτος και ονομάζει τον πίνακα Patient_Yearly_Stay. Αντίστοιχα στο δεύτερο μέρος βρίσκει από τον πρώτο πίνακα πόσοι ασθενείς έχουν τον ίδιο αριθμό νοσηλειών μεταξύ τους. Τέλος, ενώνει τους πίνακες με join ώστε να ελέ

10)

```
SELECT ms1.substance_id AS substance_id_1,
       ms2.substance_id AS substance_id_2,
       asub1.substance_name AS substance_name_1,
       asub2.substance_name AS substance_name_2,
       COUNT(DISTINCT p1.hospitalization_id) AS frequency
FROM MEDICATION_SUBSTANCE ms1
JOIN PRESCRIPTION p1 ON p1.medication_id = ms1.medication_id
JOIN PRESCRIPTION p2 ON p2.hospitalization_id = p1.hospitalization_id
JOIN MEDICATION_SUBSTANCE ms2 ON p2.medication_id = ms2.medication_id AND ms1.substance_id < ms2.substance_id
JOIN ACTIVE_SUBSTANCE asub1 ON ms1.substance_id = asub1.substance_id
JOIN ACTIVE_SUBSTANCE asub2 ON ms2.substance_id = asub2.substance_id
GROUP BY ms1.substance_id, ms2.substance_id
ORDER BY frequency DESC
LIMIT 3;
```

Σε αυτό το query βρίσκουμε τα top 3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης. Στο query γίνονται τέσσερα join 2 μεταξύ medication_substance και 2 μεταξύ prescription. Με το πρώτο join μεταξύ medication_substance και prescription αυτό που κρατάμε είναι για συνταγογράφηση όλες τις δραστικές ουσίες που περιέχει ένα φάρμακο ενώ με το δεύτερο join παίρνουμε για όλα τα ζεύγη φαρμάκων για συγκεκριμένη νοσηλεία. Έπειτα με το τρίτο join παίρνουμε για το φ2 του προαναφερθέν ζεύγους (φ1, φ2) όλες τις δραστικές ουσίες του φ2 και έτσι δημιουργούνται ζεύγοι:

δραστική ουσία_{φ1}^x = δραστική ουσία_{φ2}^y, $\forall x, y : (x \neq y) \forall \varphi1, \varphi2$:

$$\varphi_1 \neq \varphi_2 \text{ και } (\text{νοσηλεία} - id)_{\varphi_1} = (\text{νοσηλεία} - id)_{\varphi_2}$$

Με τα δύο τελευταία join παίρνουμε των από τις επιθυμητές δυνάδες ουσιών , με το ‘group by’ ενώνουμε τις ίδιες δυνάδες μεταξύ τους, για να μπορέσουμε αυτές να μετρηθούν και έπειτα με το ‘order by ... limit 3;’ παίρνουμε τις 3 συχνότερα εμφανιζόμενες δυνάδες.

11)

```
WITH doc_surgeries AS (
  SELECT sur.surgeon_id, COUNT(*) AS total_surgeries
  FROM SURGERY sur
  JOIN HOSPITALIZATION_PROCEDURE hp ON sur.surgery_id = hp.procedure_hospitalization_id
  WHERE YEAR(hp.procedure_start_time) = YEAR(CURRENT_DATE)
  GROUP BY sur.surgeon_id
)
SELECT
  s.AMKA,
  s.first_name,
  s.last_name,
  COALESCE(ds.total_surgeries, 0) AS surgeries_done
FROM STAFF s
JOIN DOCTOR d ON s.staff_id = d.staff_id
LEFT OUTER JOIN doc_surgeries ds ON s.staff_id = ds.surgeon_id
WHERE COALESCE(ds.total_surgeries, 0) <= (
  SELECT MAX(total_surgeries)-5
  FROM doc_surgeries
);
```

Σε αυτό το query βρίσκουμε όλους τους γιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

Το query1 μέσα στο ‘...left outer join (query1)...’ παίρνει κάθε χειρουργό που έχει χειρουργήσει φέτος και τον αριθμό των επεμβάσεων του. Ο προκύπτων πίνακας κάνει left outer join με το staff, παίρνοντας έτσι κάθε ζεύγος :

(γιατρός , χειρουργός_{επέμβαση φέτος}) τ. ω. : (γιατρός – id) = (χειρουργός – id)

Με την χρήση του left outer join κρατάμε και τους γιατρούς που δεν έχουν χειρουργήσει φέτος, γιατί και αυτοί θα έχουν λιγότερα από πέντε χειρουργεία από τον γιατρό με τα περισσότερα (υποθέτοντας ότι αυτός υπάρχει και έχει 5 χειρουργεία). Έπειτα με την συνθήκη του WHERE κρατάμε μόνο τους γιατρούς που έχουν λιγότερες από 4 επεμβάσεις από αυτόν με τις περισσότερες.

12)

```
SELECT
    sh.department_name,
    sh.shift_date,
    sh.slot,
    all_personnel.staff_type,
    all_personnel.subtype,
    COUNT(*) AS number_of_personnel
FROM SHIFTS sh
JOIN DUTY_TEAM dt ON sh.shift_id = dt.shift_id
JOIN
(
    SELECT staff_id, 'doctor' AS staff_type, specialty AS subtype
    FROM DOCTOR

    UNION ALL

    SELECT staff_id, 'nurse' AS staff_type, `rank` AS subtype
    FROM NURSES

    UNION ALL

    SELECT staff_id, 'administrative staff' AS staff_type, role AS subtype
    FROM ADMINISTRATIVE_STAFF
) AS all_personnel ON dt.staff_id = all_personnel.staff_id
WHERE sh.shift_date >= DATE '2026-05-18'
AND sh.shift_date < DATE '2026-05-18' + INTERVAL 7 DAY
GROUP BY
    sh.department_name,
    sh.shift_date,
    sh.slot,
    all_personnel.staff_type,
    all_personnel.subtype
ORDER BY
    sh.department_name,
    sh.shift_date,
    sh.slot,
    all_personnel.staff_type
```

Σε αυτό το query βρίσκουμε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού. Πιο συγκεκριμένα ενώνουμε τα μέλη του προσωπικού με τις βάρδιες στις οποίες συμμετέχουν μέσω του duty_teams και τα φιλτράρουμε αναφορικά με την εβδομάδα στην οποία τις δήλωσαν. Κάνουμε λοιπόν δύο join ένα μεταξύ του shifts και του duty_team και ένα άλλο join με τον πίνακα που δημιουργήσαμε ώστε να περιέχει για κάθε μέλος του προσωπικού το staff_id του και τον ρόλο του και το duty_team.

13)

```
WITH RECURSIVE supervisor_chain AS (

    SELECT
        d.staff_id AS start_doc,
        d.staff_id AS current_doc,
        d.supervisor,
        dr.description AS doctor_ranks,
        0 AS lvl
    FROM doctor d
    JOIN doctor_ranks dr
        ON dr.rank = d.rank

    UNION ALL

    SELECT
        sc.start_doc,
        d.staff_id AS current_doc,
        d.supervisor,
        dr.description AS doctor_ranks,
        sc.lvl + 1 AS lvl
    FROM supervisor_chain sc
    JOIN doctor d
        ON d.staff_id = sc.supervisor
    JOIN doctor_ranks dr
        ON dr.rank = d.rank

)

SELECT *
FROM supervisor_chain
ORDER BY start_doc, lvl;
```

Με αυτό το query θέλουμε να βρούμε όλη την αλυσίδα εποπτείας για κάθε γιατρό. Επομένως σε αυτή την περίπτωση όπως μας ζητάται και από την εκφώνηση χρησιμοποιούμε την αναδρομή. Κάθε αναδρομικό query στην SQL χωρίζεται σε δύο βασικά μέρη όπου το πρώτο ορίζει την βάση της αναδρομής και το δεύτερο που την συνεχίζει και την εκτελεί. Όπως παρατηρούμε και από τον κώδικα το πρώτο subquery βρίσκει κάθε φορά την βάση και το δεύτερο υπολογίζει την αλυσίδα εποπτείας για τον επόμενο γιατρό και η διαδικασία συνεχίζεται και εκτελείται για κάθε γιατρό. Ο κώδικας επιστρέφει έναν πίνακα στην πρώτη στήλη του οποίου φαίνεται ο γιατρός από τον οποίο ξεκινά η αλυσίδα και στην δεύτερη στήλη ο τρέχον γιατρός της αλυσίδας. Στην τρίτη και τέταρτη στήλη του πίνακα φαίνονται ο επόπτης και η βαθμίδα του τρέχοντος γιατρού.

14)

```
WITH number_of_diagnoses_per_year AS (
    SELECT
        admission_diagnosis AS diagnosis_category,
        COUNT(*) AS number_of_cases,
        YEAR(admission_date) AS diag_year
    FROM HOSPITALIZATION
    GROUP BY admission_diagnosis, YEAR(admission_date)
    HAVING COUNT(*) >= 5
)
SELECT DISTINCT nd1.diagnosis_category, nd1.diag_year AS year1, nd1.diag_year+1 AS year2, nd1.number_of_cases
FROM number_of_diagnoses_per_year nd1
WHERE EXISTS (
    SELECT 1
    FROM number_of_diagnoses_per_year nd2
    WHERE nd1.diagnosis_category = nd2.diagnosis_category
        AND nd1.number_of_cases = nd2.number_of_cases
        AND nd1.diag_year = nd2.diag_year - 1
)
```


Με το παραπάνω query θέλουμε να βρούμε τις διαγνώσεις που είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη και είχαν περισσότερα από 5 περιστατικά ανά έτος. Στην υλοποίηση μας με το πρώτο subquery βρίσκουμε τις νοσηλείες που είχαν την ίδια διάγνωση περισσότερες από πέντε φορές μέσα σε διάστημα ενός χρόνου. Στην συνέχεια με το δεύτερο subquery εντοπίζουμε τις διαγνώσεις οι οποίες είχαν στο ίδιο έτος τον ίδιο αριθμό εισαγωγών και περισσότερες από 5 νοσηλείες ανά έτος,

15)

```
WITH urgency_level_stats AS (
    SELECT
        tr.urgency_level,
        COUNT(*) AS number_of_cases,
        AVG(TIMESTAMPDIFF(MINUTE, tr.arrival_time, tr.service_time)) AS avg_wait_time,
        (COUNT(ho.hospitalization_id) * 100.0 / COUNT(*)) AS admission_percentage
    FROM triage tr
    LEFT OUTER JOIN hospitalization ho ON tr.triage_id = ho.triage_id
    GROUP BY tr.urgency_level
),
department_urg_level_stats AS (
    SELECT
        tr.urgency_level,
        ho.department_name,
        COUNT(*) AS dept_referrals
    FROM triage tr
    LEFT OUTER JOIN hospitalization ho ON tr.triage_id = ho.triage_id
    GROUP BY tr.urgency_level, ho.department_name
)
SELECT
    uls.urgency_level,
    uls.number_of_cases,
    uls.admission_percentage,
    CASE WHEN ds.department_name IS NULL THEN 'Not Admitted' ELSE ds.department_name END AS department_name,
    ds.dept_referrals
FROM urgency_level_stats uls
JOIN department_urg_level_stats ds ON uls.urgency_level = ds.urgency_level
ORDER BY uls.urgency_level ASC, ds.dept_referrals DESC;
```

Τώρα θέλουμε να δούμε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο και το ποσοστό τους που οδήγησε σε νοσηλεία. Στον κώδικα στο πρώτο μέρος βρίσκουμε στατιστικά του urgency level και με ένα join συνδέουμε την νοσηλεία με τα triage ώστε να φτιάξουμε να βρούμε ανά επίπεδο επείγοντος τον αριθμό των περιστατικών, τον μέσο χρόνο αναμονής και το ποσοστό που μας ζητάται. Το left outer join το κάνουμε γιατί θέλουμε να κρατήσουμε από το join όλο τον αριθμό των triages ώστε να υπολογιστεί το ποσοστό. Μετά κάνει το ίδιο ανά τμήμα και μεριμνά και για την περίπτωση των triages που δεν οδήγησαν σε νοσηλεία, ενώ ενώνουμε τους πίνακες που δημιουργήσαμε για να πάρουμε το τελικό αποτέλεσμα.

TRIGGERS

Εξαιτίας του γεγονότος ότι οι εισαγωγές στο σύστημα του νοσοκομείου δεν μπορούν να γίνονται με τρόπο αυθαίρετο αλλά πρέπει να τηρούνται κανόνες σε τομείς όπως οι βάρδιες και τα χειρουργεία, έχουμε υλοποιήσει πληθώρα από triggers. Τα triggers είναι μηχανισμοί οι οποίοι “αντιδρούν” όταν συμβαίνει ένα γεγονός (insert, update, delete) στην βάση μας και τρέχουν ώστε να αποτρέψουν κάποια αλλαγή ή για να διατηρήσουν ενημερωμένο κάποιο πεδίο. Προκειμένου, λοιπόν, να διατηρείται η συνέπεια των δεδομένων στην βάση έχουμε ορίσει τα triggers που παρουσιάζονται στην συνέχεια.

Για την υλοποίηση πολλών triggers αρχικά έχουμε δημιουργήσει αρκετές stored procedures, δηλαδή διαδικασίες οι οποίες καλούνται από τα triggers. Αυτό έχει γίνει διότι είναι η MySQL δεν επιτρέπει να ορίσουμε την ενεργοποίηση του trigger με την μορφή:

BEFORE INSERT OR UPDATE

Έτσι, για να μην επαναλαμβάνεται ο ίδιος κώδικας που διαχειρίζεται τις αλλαγές ή κάνει τους ελέγχους, έχουμε συχνά δύο triggers, ένα για insert και ένα για update, στα οποία καλούμε την ίδια διαδικασία. Οι βασικότερες διαδικασίες είναι οι παρακάτω:

1. Προκειμένου να εξασφαλίσουμε ότι κανένας ασθενής δεν πρόκειται να συνταγογραφηθεί φάρμακο που περιέχει κάποια ουσία στην οποία είναι αλλεργικός ορίσαμε την παρακάτω συνάρτηση. Με το query που εκτελείται βρίσκουμε αν υπάρχει κάποια τέτοια ουσία στο φάρμακο που του έχουν γράψει και σε περίπτωση που αυτό συμβαίνει προκύπτει μήνυμα σφάλματος. Η συνάρτηση αυτή θα χρησιμοποιηθεί αργότερα από triggers που θα εξασφαλίζουν την ασφάλεια των αλλεργικών ασθενών.

```
CREATE PROCEDURE check_patient_allergies(IN p_medication_id INT, IN p_amka BIGINT)
BEGIN
  IF EXISTS (
    SELECT 1
    FROM MEDICATION_SUBSTANCE ms
    JOIN ALLERGIC_PATIENTS ap ON ms.substance_id = ap.substance_id
    WHERE ms.medication_id = p_medication_id
    AND ap.AMKA = p_amka
  ) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Patient is allergic to one or more substances in this drug';
  END IF;
END$$
```

2. Με αυτό το procedure υπολογίζουμε για κάθε μέλος του προσωπικού οποιοσδήποτε κι αν είναι ο ρόλος το πόσες βάρδιες έχει πραγματοποιήσει ανά μήνα. Στην συνέχεια με βάση τον ρόλο του καθενός γίνεται έλεγχος για το αν ξεπερνάει το μέγιστο αριθμό ανά μήνα. Στην περίπτωση που αυτό συμβαίνει η συνάρτηση δίνει μήνυμα σφάλματος και επομένως οποιαδήποτε άκυρη εισαγωγή σε αυτό το κομμάτι δεν πραγματοποιείται.

```

CREATE PROCEDURE check_monthly_duty_limits(IN p_staff_id INT, IN p_shift_id INT)
BEGIN
    DECLARE staff_role VARCHAR(25);
    DECLARE new_shift_date DATE;
    DECLARE counter INT;

    SELECT staff_type INTO staff_role
    FROM STAFF
    WHERE staff_id = p_staff_id;

    SELECT shift_date INTO new_shift_date
    FROM SHIFTS
    WHERE shift_id = p_shift_id;

    SELECT COUNT(*) INTO counter
    FROM DUTY_TEAM d
    JOIN SHIFTS s ON d.shift_id = s.shift_id
    WHERE d.staff_id = p_staff_id
    AND MONTH(s.shift_date) = MONTH(new_shift_date)
    AND YEAR(s.shift_date) = YEAR(new_shift_date);

    IF staff_role = 'doctor' AND counter >= 15 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Every doctor has to do only 15 shifts per month';
    END IF;

    IF staff_role = 'nurse' AND counter >= 20 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Every nurse has to do only 20 shifts per month';
    END IF;

    IF staff_role = 'administrative staff' AND counter >= 25 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'The administrative staff has to do only 25 shifts per month';
    END IF;
END$$

```

3. Σε άμεση αντιστοιχία με την προηγούμενη, η διαδικασία `check_continuous_night_shifts` ελέγχει αν κάποιο μέλος του προσωπικού έχει δηλωθεί σε δύο νυχτερινές βάρδιες δύο διαδοχικών ημερών.

```

CREATE PROCEDURE check_continuous_night_shifts(IN p_staff_id INT, IN p_shift_id INT)
BEGIN
    DECLARE new_shift_date DATE;
    DECLARE is_night INT;
    DECLARE cnt INT;

    SELECT shift_date,
           CASE WHEN slot = 'night' THEN 1 ELSE 0 END
    INTO new_shift_date, is_night
    FROM SHIFTS
    WHERE shift_id = p_shift_id;

    IF is_night = 1 THEN
        SELECT COUNT(*) INTO cnt
        FROM DUTY_TEAM dt
        JOIN SHIFTS s ON dt.shift_id = s.shift_id
        WHERE dt.staff_id = p_staff_id
        AND s.slot = 'night'
        AND s.shift_date < new_shift_date
        AND s.shift_date >= DATE_SUB(new_shift_date, INTERVAL 3 DAY);

        IF cnt >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Max 3 consecutive night shifts exceeded';
        END IF;
    END IF;
END$$

```

4. Με την διαδικασία `check_eight_hours` ελέγχουμε αν υπάρχει κάποιος εργαζόμενος δηλωμένος γενικά σε δύο συνεχόμενες βάρδιες. Οι τρεις τελευταίες διαδικασίες είναι υπεύθυνες για την τήρηση των κανόνων της εισαγωγής στις βάρδιες. Όπως θα δούμε και

αργότερα θα κληθούν από τα triggers που θα μεριμνούν για την σωστή εισαγωγή των δεδομένων στον πίνακα duty_staff που περιλαμβάνει τις αναθέσεις βαρδιών στα μέλη του προσωπικού.

```
CREATE PROCEDURE check_eight_hours_rest(IN p_staff_id INT, IN p_shift_id INT)
BEGIN
    DECLARE ns DATETIME;

    SELECT TIMESTAMP(
        shift_date,
        CASE slot
            WHEN 'morning' THEN '07:00:00'
            WHEN 'afternoon' THEN '15:00:00'
            WHEN 'night' THEN '23:00:00'
        END
    )
    INTO ns
    FROM SHIFTS
    WHERE shift_id = p_shift_id;

    IF EXISTS (
        SELECT 1
        FROM DUTY_TEAM dt
        JOIN SHIFTS s ON s.shift_id = dt.shift_id
        WHERE dt.staff_id = p_staff_id
        AND dt.shift_id != p_shift_id
        AND ABS(
            TIMESTAMPDIFF(
                HOUR,
                TIMESTAMP(
                    s.shift_date,
                    CASE s.slot
                        WHEN 'morning' THEN '07:00:00'
                        WHEN 'afternoon' THEN '15:00:00'
                        WHEN 'night' THEN '23:00:00'
                    END
                ),
                ns
            ) < 16
        ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = '8 hours rest required between shifts';
    END IF;
END$$
```

5. Προκειμένου να ελέγχουμε αν μια βάρδια έχει τον απαραίτητο αριθμό προσωπικού ώστε να λειτουργεί ορθά έχουμε τοποθετήσει στον πίνακα shifts ένα attribute το οποίο δείχνει αν η βάρδια είναι πλήρης ή όχι. Η επόμενη διαδικασία καλείται προκειμένου να ανανεώνει το αν την κάθε βάρδια αν κάποια εισαγωγή στο duty_teams την κάνει πλήρη.

```
CREATE PROCEDURE auto_update_shift_status(IN p_shift_id INT)
BEGIN
    DECLARE doc_count INT;
    DECLARE nurse_count INT;
    DECLARE admin_count INT;
    DECLARE resident_count INT;
    DECLARE higher_rank_count INT;

    SELECT
        COALESCE(SUM(CASE WHEN s.staff_type = 'doctor' THEN 1 ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN s.staff_type = 'nurse' THEN 1 ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN s.staff_type = 'administrative staff' THEN 1 ELSE 0 END), 0)
    INTO doc_count, nurse_count, admin_count
    FROM DUTY_TEAM t
    JOIN STAFF s ON t.staff_id = s.staff_id
    WHERE t.shift_id = p_shift_id;

    SELECT
        COALESCE(SUM(CASE WHEN d.rank = 1 THEN 1 ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN d.rank IN (3, 4) THEN 1 ELSE 0 END), 0)
    INTO resident_count, higher_rank_count
    FROM DUTY_TEAM t
    JOIN DOCTOR d ON d.staff_id = t.staff_id
    WHERE t.shift_id = p_shift_id;

    IF doc_count >= 3
    AND nurse_count >= 6
    AND admin_count >= 2
    AND (resident_count = 0 OR higher_rank_count > 0) THEN
        UPDATE SHIFTS
        SET shift_status = 'full'
        WHERE shift_id = p_shift_id
        AND shift_status = 'not_full';
    END IF;
END
```

```

ELSE
    UPDATE SHIFTS
    SET shift_status = 'not full'
    WHERE shift_id = p_shift_id
    AND shift_status = 'full';
END IF;
END$$

```

6. Εδώ ξεκινάμε με το πρώτο μας trigger το οποίο ελέγχει αν για κάποια εισαγωγή στον πίνακα doctor, ο επόπτης που προστέθηκε είχε rank μικρότερο από το ίδιο του γιατρό που εισήχθη στην βάση. Γενικά στην βάση μας έχουμε δεχθεί ότι ένας γιατρός μπορεί να εποπτεύει μόνο κάποιον κατώτερου βάρθμου.

```

CREATE TRIGGER supervision_chain
BEFORE INSERT ON DOCTOR
FOR EACH ROW
BEGIN
    DECLARE sup_rank INT;

    IF NEW.supervisor IS NOT NULL THEN
        SELECT `rank` INTO sup_rank
        FROM DOCTOR
        WHERE staff_id = NEW.supervisor;

        IF NEW.`rank` >= sup_rank THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Supervisor must have higher rank';
        END IF;
    END IF;
END$$

```

7. Επειδή κάθε τμήμα έχει ως attribute τον αριθμό των κρεβατιών που διαθέτει θέλουμε αυτός ο αριθμός να είναι πάντα επικαιροποιημένος όσες εισαγωγές και να κάνουμε στα beds. Αυτό εξασφαλίζεται με το παρακάτω trigger το οποίο κάθε φορά που γίνεται μια εισαγωγή στις κλίνες ενημερώνει τον πίνακα των τμημάτων με την νέα τιμή των κρεβατιών. Αντίστοιχη μέριμνα υπάρχει και στην περίπτωση διαγραφής μιας κλίνης.

```

CREATE TRIGGER beds_inc
AFTER INSERT ON BEDS
FOR EACH ROW
BEGIN
    UPDATE DEPARTMENT
    SET bed_count = bed_count + 1
    WHERE department_name = NEW.department_name;
END$$

CREATE TRIGGER beds_dec
AFTER DELETE ON BEDS
FOR EACH ROW
BEGIN
    UPDATE DEPARTMENT
    SET bed_count = bed_count - 1
    WHERE department_name = OLD.department_name;
END$$

```

8) Στα επόμενα trigger χρησιμοποιούμε τις διαδικασίες που ορίσαμε προηγουμένως ώστε κάθε φορά που πάει να γίνει κάποιο insert στην συνταγογράφηση να ελέγχεται αν το φάρμακο που πρόκειται να λάβει ο ασθενής περιέχει κάποια ουσία στην οποία είναι αλλεργικός.

```
CREATE TRIGGER allergic_drugs_ins
BEFORE INSERT ON PRESCRIPTION
FOR EACH ROW
BEGIN
    CALL check_patient_allergies(NEW.medication_id, NEW.AMKA);
END$$

CREATE TRIGGER allergic_drugs_upd
BEFORE UPDATE ON PRESCRIPTION
FOR EACH ROW
BEGIN
    CALL check_patient_allergies(NEW.medication_id, NEW.AMKA);
END$$
```

9) Με αυτό το trigger τώρα ελέγχουμε το αν ο ασθενής έχει φύγει από το νοσοκομείο την στιγμή που πραγματοποιεί μια αξιολόγηση για την νοσηλεία του.

```
CREATE TRIGGER evaluation_hosp
BEFORE INSERT ON EVALUATION_HOSPITALIZATION
FOR EACH ROW
BEGIN
    DECLARE d_date DATE;

    SELECT h.discharge_date INTO d_date
    FROM HOSPITALIZATION h
    WHERE h.hospitalization_id = NEW.hospitalization_id;

    IF d_date IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'You cannot evaluate before you leave the hospital';
    END IF;
END$$
```

10) Με το επόμενο trigger πραγματοποιούμε έλεγχο για τα inserts στον πίνακα doctor evaluation. Πιο συγκεκριμένα δεν επιτρέπουμε αξιολογήσεις ασθενών οι οποίοι δεν έχουν ακόμα φύγει από το νοσοκομείο ή που δεν έχουν λάβει κάποια συνταγογράφηση από τον γιατρό που θέλουν να αξιολογήσουν.

```
CREATE TRIGGER evaluation_doctor
BEFORE INSERT ON DOCTOR_EVALUATION
FOR EACH ROW
BEGIN
    DECLARE d_date DATE;
    DECLARE patient_amka BIGINT;

    SELECT t.AMKA, h.discharge_date
    INTO patient_amka, d_date
    FROM HOSPITALIZATION h
    JOIN TRIAGE t ON h.triage_id = t.triage_id
    WHERE h.hospitalization_id = NEW.hospitalization_id;

    IF NOT EXISTS (
        SELECT 1
        FROM PRESCRIPTION p
        WHERE p.staff_id = NEW.doctor_id
        AND p.AMKA = patient_amka
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'You cannot evaluate a doctor that did not prescribe you';
    END IF;

    IF d_date IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'You cannot evaluate a doctor before you leave';
    END IF;
END$$
```

11) Με τα επόμενα triggers ελέγχουμε το αν τηρούνται όλοι οι κανόνες εισαγωγής των γιατρών σε βάρδιες χρησιμοποιώντας τις συναρτήσεις που υλοποιήσαμε στην αρχή.

```
CREATE TRIGGER duty_limit_monthly_ins
BEFORE INSERT ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL check_monthly_duty_limits(NEW.staff_id, NEW.shift_id);
END$$

CREATE TRIGGER duty_limit_monthly_upd
BEFORE UPDATE ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL check_monthly_duty_limits(NEW.staff_id, NEW.shift_id);
END$$

CREATE TRIGGER duty_limit_continuous_nights_ins
BEFORE INSERT ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL check_continuous_night_shifts(NEW.staff_id, NEW.shift_id);
END$$

CREATE TRIGGER duty_limit_continuous_nights_upd
BEFORE UPDATE ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL check_continuous_night_shifts(NEW.staff_id, NEW.shift_id);
END$$
```

```
CREATE TRIGGER duty_rest_ins
BEFORE INSERT ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL check_eight_hours_rest(NEW.staff_id, NEW.shift_id);
END$$

CREATE TRIGGER duty_rest_upd
BEFORE UPDATE ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL check_eight_hours_rest(NEW.staff_id, NEW.shift_id);
END$$
```

12) Με το επόμενο trigger ελέγχουμε το αν μετά από κάποιο update στον πίνακα τον βαρδιών η κατάσταση τους παραμένει ενημερωμένη και σωστή.

```
CREATE TRIGGER valid_team
BEFORE UPDATE ON SHIFTS
FOR EACH ROW
BEGIN
    DECLARE doc_count INT;
    DECLARE nurse_count INT;
    DECLARE admin_count INT;
    DECLARE resident_count INT;
    DECLARE higher_rank_count INT;

    IF NEW.shift_status = 'full' THEN
        SELECT
            COALESCE(SUM(CASE WHEN s.staff_type = 'doctor' THEN 1 ELSE 0 END), 0),
            COALESCE(SUM(CASE WHEN s.staff_type = 'nurse' THEN 1 ELSE 0 END), 0),
            COALESCE(SUM(CASE WHEN s.staff_type = 'administrative staff' THEN 1 ELSE 0 END), 0)
        INTO doc_count, nurse_count, admin_count
        FROM DUTY_TEAM t
        JOIN STAFF s ON t.staff_id = s.staff_id
        WHERE t.shift_id = NEW.shift_id;

        IF doc_count < 3 OR nurse_count < 6 OR admin_count < 2 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Invalid shift team composition';
        END IF;

        SELECT
            COALESCE(SUM(CASE WHEN d.rank = 1 THEN 1 ELSE 0 END), 0),
            COALESCE(SUM(CASE WHEN d.rank IN (3, 4) THEN 1 ELSE 0 END), 0)
        INTO resident_count, higher_rank_count
        FROM DUTY_TEAM t
        JOIN DOCTOR d ON d.staff_id = t.staff_id
        WHERE t.shift_id = NEW.shift_id;
```

```

        IF resident_count > 0 AND higher_rank_count = 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'No high ranking doctor when resident present';
        END IF;
    END IF;
END$$

```

13) Με τα επόμενα triggers χρησιμοποιούμε και πάλι τις συναρτήσεις που γράψαμε προηγουμένως έτσι ώστε μετά από κάθε insert και update στον πίνακα του duty_teams να ενημερώνεται το shift status σωστά με βάση τον αριθμό των γιατρών που έχουν αναλάβει να στελεχώσουν την κάθε βάρδια.

```

CREATE TRIGGER auto_shift_full_ins
AFTER INSERT ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL auto_update_shift_status(NEW.shift_id);
END$$

CREATE TRIGGER auto_shift_full_upd
AFTER UPDATE ON DUTY_TEAM
FOR EACH ROW
BEGIN
    CALL auto_update_shift_status(NEW.shift_id);

    IF OLD.shift_id != NEW.shift_id THEN
        CALL auto_update_shift_status(OLD.shift_id);
    END IF;
END$$

```

14) Σε αυτό το σημείο ορίζουμε άλλη μια διαδικασία η οποία υπολογίζει το συνολικό κόστος νοσηλείας ενός ασθενή στην περίπτωση που μένει στο νοσοκομείο περισσότερο καιρό από αυτόν που ορίζεται στο KEN. Πιο συγκεκριμένα το συνολικό κόστος προκύπτει από το κόστος KEN και το άθροισμα του μέσου όρου κάθε επιπλέον μέρας που βγαίνει πάλι με βάση το KEN.

```

CREATE PROCEDURE calculate_hospitalization_cost(
    IN p_drg_code VARCHAR(50),
    IN p_admission_date DATE,
    IN p_discharge_date DATE,
    OUT p_calculated_cost INT
)
BEGIN
    DECLARE stay_time INT;
    DECLARE mdn_cost INT;
    DECLARE mdn INT;

    SELECT average_length_of_stay, cost
    INTO mdn, mdn_cost
    FROM DRG
    WHERE drg_code = p_drg_code;

    IF p_discharge_date IS NOT NULL THEN
        SET stay_time = DATEDIFF(p_discharge_date, p_admission_date);

        IF stay_time > mdn THEN
            SET p_calculated_cost = mdn_cost + (mdn_cost / mdn) * (stay_time - mdn);
        ELSE
            SET p_calculated_cost = mdn_cost;
        END IF;
    ELSE
        SET p_calculated_cost = mdn_cost;
    END IF;
END$$

```


Ταυτόχρονα με το επόμενο trigger βρίσκουμε την τιμή του κόστους καλώντας την παραπάνω συνάρτηση για κάθε νέα εισαγωγή στον πίνακα των νοσηλειών.

```
CREATE TRIGGER final_cost_ins
BEFORE INSERT ON HOSPITALIZATION
FOR EACH ROW
BEGIN
    DECLARE temp_cost INT;

    CALL calculate_hospitalization_cost(NEW.drg_code, NEW.admission_date, NEW.discharge_date, temp_cost);

    SET NEW.current_cost = temp_cost;
END$$
```

15) Με αυτό το trigger ελέγχουμε για κάθε εισαγωγή στον πίνακα των νοσηλειών ελέγχουμε αν το κρεβάτι που δηλώνεται ότι θα βρίσκεται ο ασθενής είναι κατειλημένο ή όχι. Αν δεν είναι ορίζει πλέον την κατάσταση της κλίνης ως occupied και εκτελεί κανονικά την εισαγωγή. Εντελώς αντίστοιχο trigger έχει φτιαχτεί και για την περίπτωση του update.

```
CREATE TRIGGER ensure_bed_is_free
BEFORE INSERT on HOSPITALIZATION
FOR EACH ROW BEGIN
    declare bed_stat CHAR(30);
    IF new.discharge_date is NULL OR new.discharge_date > CURRENT_DATE()
    THEN
        select status
        into bed_stat
        from BEDS
        where NEW.bed_id = bed_number;

        IF bed_stat <> 'not occupied' then
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'This bed is not available for hospitalization';
        END IF;

        UPDATE BEDS
        set status = 'occupied'
        WHERE bed_number = new.bed_id ;
    END IF;
END$$
```

16) Εδώ θέλουμε να βεβαιωθούμε ότι δεν θα γίνει κάποια εισαγωγή μέλους του προσωπικού που δεν είναι χειρουργός ως υπεύθυνος χειρουργός σε κάποια επέμβαση. Επομένως έχουμε φτιάξει μια stored procedure που να κάνει αυτό ακριβώς το πράγμα και ένα trigger που κάθε φορά που γίνεται insert στον πίνακα surgery να την καλεί και να ελέγχει τη συνθήκη. Αντίστοιχη μέριμνα έχει ληφθεί και εδώ για την περίπτωση του update με ένα πανομοιότυπο δεύτερο trigger.

```

CREATE PROCEDURE check_is_surgeon (IN p_surgeon_id INT)
BEGIN
    declare spec char(40);
    select specialty
    into spec
    FROM DOCTOR
    where staff_id = p_surgeon_id;

    IF spec NOT IN ('Neurosurgeon', 'Gynecology') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Only surgeons can do surgeries';
    END IF;
END$$

CREATE TRIGGER is_surgeon_insert
BEFORE INSERT ON SURGERY
FOR EACH ROW
BEGIN
    CALL check_is_surgeon(NEW.surgeon_id);
END$$

```

17) Με αυτά τα δύο triggers τόσο μετά από insert όσο και μετά από update υπολογίζουμε την διάρκεια του χειρουργείου ως την διαφορά μεταξύ της ώρας που ολοκληρώνεται και της ώρας όπου ξεκίνησε

```

CREATE TRIGGER trg_hosp_proc_duration_before_insert
BEFORE INSERT ON HOSPITALIZATION_PROCEDURE
FOR EACH ROW
BEGIN
    SET NEW.procedure_duration =
        TIMESTAMPDIFF(MINUTE, NEW.procedure_start_time, NEW.procedure_end_time);
END$$

CREATE TRIGGER trg_hosp_proc_duration_before_update
BEFORE UPDATE ON HOSPITALIZATION_PROCEDURE
FOR EACH ROW
BEGIN
    SET NEW.procedure_duration =
        TIMESTAMPDIFF(MINUTE, NEW.procedure_start_time, NEW.procedure_end_time);
END$$

```

18) Με το επόμενο trigger φροντίζουμε να μην μπορεί να γίνει επικάλυψη μεταξύ δύο ιατρικών πράξεων στην ίδια αίθουσα την ίδια ώρα και το αποτρέπει σε περίπτωση που συμβεί δίνοντας μήνυμα σφάλματος.

```

CREATE TRIGGER multiple_hospital_procedures_in_one_room
BEFORE INSERT ON HOSPITALIZATION_PROCEDURE
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM HOSPITALIZATION_PROCEDURE h
        WHERE h.room_id = NEW.room_id
        AND NEW.procedure_start_time < h.procedure_end_time
        AND NEW.procedure_end_time > h.procedure_start_time
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Two procedures cannot happen in the same room at the same time';
    END IF;
END$$

```

19) Τέλος με αυτό το τελευταίο trigger μεριμνούμε ώστε να μην γίνεται ένας γιατρός να δηλωθεί ότι συμμετέχει σε δύο διαφορετικά χειρουργεία την ίδια ώρα και μέρα.

```
CREATE TRIGGER one_surgeon_two_places_at_the_same_time
BEFORE INSERT ON SURGERY
FOR EACH ROW
BEGIN
    DECLARE new_start_time DATETIME;
    DECLARE new_end_time DATETIME;

    SELECT hp.procedure_start_time, hp.procedure_end_time
    INTO new_start_time, new_end_time
    FROM HOSPITALIZATION_PROCEDURE hp
    WHERE hp.procedure_hospitalization_id = NEW.surgery_id;

    IF EXISTS (
        SELECT 1
        FROM SURGERY s
        JOIN HOSPITALIZATION_PROCEDURE hp
        ON hp.procedure_hospitalization_id = s.surgery_id
        WHERE s.surgeon_id = NEW.surgeon_id
        AND new_start_time < hp.procedure_end_time
        AND new_end_time > hp.procedure_start_time
        AND s.surgery_id <> NEW.surgery_id
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'One doctor cannot be in two rooms at the same time';
    END IF;
END$$
```

User Interface

Το site αποτελεί μια web εφαρμογή διαχείρισης βάσης δεδομένων του νοσοκομείου “Υγειόπολης”. Το site υλοποιήθηκε με PHP και MySql και εκτελείται μέσω του XAMPP σε περιβάλλον localhost. Μέσω της εφαρμογής είναι δυνατή η προβολή , η ενημέρωση και διαγραφή των δεδομένων από όλα τα tables της βάσης.

Η πρώτη σελίδα που φορτώνεται είναι η index.php η οποία αρχικά δείχνει τον αριθμό των ασθενών που δεν έχουν εισαχθεί ακόμη για διαλογή στο νοσοκομείο. Ακόμη, στην σελίδα εμφανίζονται οι διαθέσιμοι πίνακες μέσω αλληλεπίδρασης με την βάση και μέσα σε κάθε block πίνακα υπάρχει ένα κουμπί “button” που στέλνει τον χρήστη στον αντίστοιχο .php αρχείο του πίνακα .Ακόμη, για κάθε πίνακα εμφανίζεται ο αριθμός εγγραφών που περιέχει.

Η σύνδεση με την βάση γίνεται μέσω του “db_connections.php”. Ένα ακόμη βοηθητικό αρχείο είναι το “table_helpers.php” με αρκετές βοηθητικές συναρτήσεις. Αυτό το αρχείο χρησιμοποιήθηκε μόνο για το “index.php” και το “queries.php”.

Το “queries.php” υλοποιεί το site για την χρήση των queries χωρίς όμως να διαθέτει τα ίδια τα queries μέσα στο κώδικα του. Ο sql κώδικας των queries εμπεριέχεται μέσα στο “query_definitions.php” το οποίο είναι απλά ένα αρχείο με ένα πολυδιάστατο array το οποίο αποθηκεύει για κάθε query:

- τον αριθμό του με βάση την εκφώνηση της εξαμηνιαίας εργασίας
- Ένα μικρό caption που περιγράφει την λειτουργία
- Τον κώδικα sql του query

Έπειτα κάθε table της βάσης έχει τρία αρχεία που του αντιστοιχούν:

- <table>.php
- create_<table>.php
- update_<table>.php
- delete_<table>.php

Κάθε αρχείο που ανήκει σε μία από τις τρεις κατηγορίες δεν χρησιμοποιεί κάποιο βοηθητικό .php αρχείο πέρα από το “db_connections.php” για να συνδεθεί με την βάση. Ο μόνος τρόπος για να αλλάξει ο χρήστης τα δεδομένα της βάσης είναι να περάσει στο επιθυμητό table μέσω του αρχείου <table.php>.

Για την υλοποίηση του UI χρησιμοποιήθηκαν LLMS.